Leçon 4 : Modèle logique de données relationnel

Dr SANGARE Seydou

Table des matières

Objectifs 3

Introduction 4

I - Quels sont les concepts de bases du modèle relationnel ? 5

1. Notion d'attribut et de Domaine : 5

2. Notion de Produit cartésien : 6

3. Notion de Relation 7

3.1. Sous l'angle mathématique (définition formelle) 7

3.2. Sous l'angle pragmatique (représentation concrète) 7

4. Notion de clé de relation 9

II - Exercice 11

III - Exercice 12

IV - Quelles sont les contraintes d'intégrité ? 13

V - Exercice 15

VI - Exercice 16

VII - Qu'est-ce que la dépendance fonctionnelle ? 17

1. Dépendances Fonctionnelles (DF) 17

VIII - Exercice 21

IX - Exercice 22

X - Qu'est-ce que la normalisation? 23

1. Première Forme Normale (1FN) 24

2. Deuxième Forme Normale (2FN) 24

3. Troisième Forme Normale (3FN) 25

XI - Exercice 26

XII - Exercice 27

XIII - Comment transformer le MCD en Modèle Logique de Données Relationnel (MLDR) ? 28

XIV - Exercice 31

XV - Exercice 32

Conclusion 33

Objectifs

A la fin de ce cours, les participants seront capables de :

- Définir les concepts de base d'un modèle logique de données relationnel;
- Concevoir un modèle relationnel ;
- Transformer un MCD en un modèle logique de données relationnel.

Introduction

Le modèle relationnel constitue la base théorique et pratique des systèmes de gestion de bases de données modernes (SGBDR).

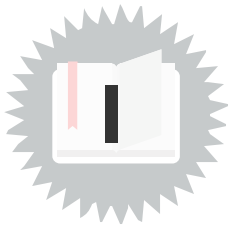
Introduit par E.F. Codd en 1970, il repose sur la théorie des ensembles et la logique relationnelle.

Son objectif est de représenter les données de manière structurée à travers des relations (ou tables), composées de colonnes (attributs) et de lignes (tuples).

Cette leçon présente les concepts fondamentaux du modèle relationnel, les contraintes d'intégrité, les dépendances fonctionnelles, la normalisation des relations et la transformation du MCD en MLDR.

Elle constitue une étape cruciale pour tout concepteur de base de données cherchant à assurer la cohérence, la fiabilité et l'efficacité des systèmes d'information.

Quels sont les concepts de bases du modèle relationnel ?



1. Notion d'attribut et de Domaine :

Un attribut est la plus petite unité d'information significative dans une base de données relationnelle. Il représente une caractéristique ou une propriété d'une entité que l'on souhaite modéliser dans un système d'information ou une base de données. Dans le contexte d'une table, un attribut correspond à une colonne. Chaque attribut correspond à une information précise qui contribue à la représentation complète de l'entité. Par exemple, dans la modélisation d'une entité ***Personne***, des attributs sont :

- **nom**
- **prenom**
- **DateNaissance**
- **Adresse**
- **Email**

Ces informations permettent d'identifier et de caractériser la personne de manière unique ou spécifique.

Le domaine d'un attribut correspond à l'ensemble de toutes les valeurs possibles que cet attribut peut prendre. Ce domaine peut être :

- **fini**, par exemple un ensemble limité de valeurs discrètes (exemples : {rouge, bleu, vert} pour l'attribut "**CouleurPreferee**")
- **infini**, comme l'ensemble des entiers ou des nombres réels (exemple : attribut "**Age**" pouvant être 0, 1, 2, ..., etc.)

Plusieurs attributs peuvent partager le même domaine si leurs valeurs admissibles sont du même type. Par exemple, les attributs PrixUnitaire et PrixTotal pourraient tous deux appartenir au domaine des nombres réels positifs.

Exemples:

- Attribut **age** : son domaine est l'ensemble des entiers naturels, généralement compris entre 0 et 120.
- Attribut **estEmploye** : son domaine est un ensemble booléen {vrai, faux}, indiquant si un individu est employé ou non.
- Attribut **CouleurPreferee** : son domaine peut être un ensemble fini comme {rouge, bleu, vert, jaune}.

Chaque attribut est défini sur un domaine, ce qui garantit la cohérence et l'intégrité des données.

Exemples de domaine :

Domaine	Description
`Entier`	Nombres entiers (ex : 1, 10, -5)
`Réel`	Nombres décimaux (ex : 3.14, -0.8)
`Chaîne de caractères`	Texte ou suite de caractères (ex : "Nom", "Adresse")
`Date`	Valeurs calendaires (ex : 2025-06-26)

2. Notion de Produit cartésien :

Le produit cartésien est une opération fondamentale dans le modèle relationnel. Il consiste à combiner toutes les valeurs de plusieurs ensembles de domaines pour former des n-uplets.

Définition :

Le produit cartésien d'un ensemble de domaines D_1, D_2, ..., D_n noté $D_1 \times D_2 \times ... \times D_n$ est l'ensemble de tous les n-uplets ou tuples de la forme :

$(d_1, d_2, ..., d_n)$ tels que $d_i \in D_i$ pour i=1,2,...,n. C'est-à-dire que l'on forme tous les combinaisons possibles en prenant un élément de chaque domaine.

Exemple :

Soit :

- D_1={BD, Math, IA} donc card(D_1) = 3
- D_2={3, 2} donc card(D_2) = 2

Le produit cartésien : $D_1 \times D_2$={(BD,3), (BD,2), (Math,3), (Math,2), (IA,3), (IA,2)}

D_1	D_2	$D_1 x D_2$
BD	3	(BD,3)
BD	2	(BD, 2)
MAHT	3	(MATH, 3)
MATH	2	(MATH, 2)
IA	3	(IA, 3)
IA	2	(IA, 2)

Donc, cardinalité de $D_1 \times D_2 = 3 \times 2 = 6$

3. Notion de Relation

Une **relation** en modélisation de données peut être comprise selon deux points de vue complémentaires : l'angle mathématique (sa définition formelle) et l'angle pragmatique (sa représentation concrète).

3.1. Sous l'angle mathématique (définition formelle)

Définition : Une relation est un sous-ensemble du **produit cartésien** de plusieurs domaines..

Soient :

- des attributs $A_1, A_2,...,A_n$
- leurs domaines respectifs $D_1, D_2, ..,D_n$

Une relation R est un ensemble de n-uplets (ou tuples) $(x_1,x_2,...,x_n)$ où chaque $x_i \in D_i$, pour $1 \leq i \leq n1$.

Autrement dit, une relation est une table composée de lignes (les n-uplets) et de colonnes (les attributs), où chaque ligne respecte les types de données imposés par les domaines.

Exemples

Soient :

- D_1={BD, Math, IA}
- D_2={3, 2}

Le produit cartésien complet est: $D_1 \times D_2$={(BD, 3), (BD, 2), (Math, 3), (Math, 2), (IA, 3), (IA, 2)}

Une relation est un sous-ensemble de ce produit cartésien, par exemple :

R1={(BD, 3), (Math,2)} ou encore R2={(BD, 3), (IA,3)}

3.2. Sous l'angle pragmatique (représentation concrète)

Définition :

Une relation est représentée de manière pragmatique par une **table** à deux dimensions qui organise les données en lignes et en colonnes. Les colonnes correspondent aux attributs et les lignes aux occurrences. Chaque relation représente un type d'entité ou une association entre entités.

- **Les colonnes (les attributs) :** Ce sont les caractéristiques qui décrivent les entités que l'on souhaite enregistrer. Chaque colonne a un nom unique et est associée à un domaine (un type de données : nombre, texte, date, etc.).
- **Les lignes (les n-uplets ou tuples) :** Chaque ligne représente une **occurrence** unique d'une entité, c'est-à-dire un enregistrement concret. Elle contient une valeur pour chaque attribut (qui peut parfois être `NULL` si l'information est inconnue ou non applicable).

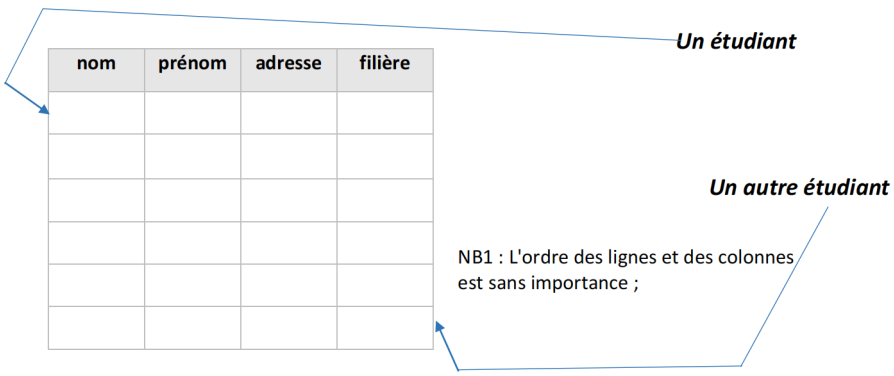


Image 1 Figure 1 : une relation étudiant

Exemple de relation : `Etudiant`

1. **Définition en intention** (structure de la relation)

Il s'agit de la **description du schéma** de la relation, c'est-à-dire des **attributs** (ou colonnes) et leur **type de données** :

`Etudiant(NCE: entier, Nom: chaîne, Prénom: chaîne, DateNaissance: date, Niveau: chaîne)`

- **NCE** : Numéro de carte d'étudiant (entier)
- **Nom** : Nom de famille (chaîne de caractères)
- **Prénom** : Prénom (chaîne de caractères)
- **DateNaissance** : Date de naissance (type date)
- **Niveau** : Niveau d'étude (ex. L1, L2, L3)

2. **Définition en extension** (contenu de la relation)

C'est un **ensemble de tuples** (lignes ou enregistrements ou occurrences), représentant les **valeurs** concrètes des étudiants inscrits. Un tuple est une instance unique d'une relation. Il représente un ensemble de valeurs pour tous les attributs de la relation, correspondant à une entité spécifique.

NCE	Nom	Prenom	DateNaissance	Niveau
1001	Koné	Awa	2002-05-12	L2
1002	Traoré	Mamadou	2001-10-20	L1
1003	Diabaté	Salif	2000-03-15	L3

Dans cet exemple, la structure entière est la relation Etudiant. Chaque ligne, par exemple (101, Koné, Awa, 2002-05-12, L2)) est un tuple ou un enregistrement ou une occurrence et NCE, Nom , Prenom , DateNaissance, Niveau sont les attributs ou colonnes.

Remarques importantes sur une relation relationnelle

1. **Pas de doublons** :

   Une relation ne peut pas contenir deux lignes (tuples) identiques.

2. **Ordre des lignes sans importance** :

   Les lignes peuvent être réarrangées sans que la relation change.

3. **Ordre des colonnes sans importance** :

   L'ordre des attributs (colonnes) n'est pas significatif en soi, même s'il peut l'être dans une présentation.

4. **Atomicité** :

   Chaque case contient **une seule valeur** (pas de liste ou tableau).

Autres définitions utiles :

- **La cardinalité** est le nombre de tuples (lignes) dans la relation. Ici : 3 (car 3 étudiants).
- **Le Degré** est le nombre d'attributs (colonnes) dans la relation. Ici : 5 (`NCE`, `Nom`, `Prénom`, `DateNaissance`, `Niveau`).

4. Notion de clé de relation

Définition : Une clé de relation est un attribut ou un ensemble minimal d'attributs dont les valeurs identifient de manière unique chaque occurrence de la relation. Minimal signifie qu'aucun sous-ensemble de cette clé ne peut identifier les tuples de manière unique. La clé sert donc à identifier sans ambiguïté un enregistrement précis dans une table.

Clé candidate :

Une **clé candidate** est une clé qui pourrait potentiellement être choisie comme clé principale. Autrement dit, c'est toute combinaison minimale d'attributs capable d'identifier une occurrence. Une relation peut posséder plusieurs clés candidates.

Clé primaire :

Parmi toutes les clés candidates, une seule est sélectionnée comme **clé primaire** de la relation. Cette clé primaire est utilisée essentiellement pour identifier les occurrences de manière unique et pour créer des liens entre tables dans une base de données relationnelle.

Exemple :

Dans la relation Etudiant, un attribut comme **NCE** (numéro d'étudiant) peut servir de clé primaire car chaque étudiant possède un numéro unique qui le distingue des autres.

Clé étrangère (ou identifiant externe) :

Une **clé étrangère** est un attribut (ou un ensemble d'attributs) dans une relation qui sert à **référencer les occurrences d'une autre relation** (ou parfois la même relation). La clé étrangère permet d'établir un lien entre deux tables différentes.

- La valeur de la clé étrangère doit toujours correspondre à la valeur de la clé primaire dans la relation référencée.
- Ceci garantit la **consistance référentielle** : on ne peut pas avoir de valeur dans la clé étrangère qui ne corresponde pas à un tuple existant dans la table cible.

Exemple :

Considérons les relations suivantes :

Relation Etudiant

NCE (clé primaire)	**Nom**	**Prénom**
100	Coulibaly	Mamadou
101	Sery	Dorcas
102	Yao	Blandine

Relation Cours

codeCours (clé primaire)	**nomCours**	**crédits**
INF101	Informatique	4
MAT201	Mathématiques	5

Relation Inscription

NCE (clé étrangère)	codeCours (clé étrangère)	dateInscription
100	INF101	2025-09-01
102	MAT201	2025-09-02
103	INF101	2025-09-01

Explications détaillées :

Clé primaire :

- Dans la relation *Étudiant*, l'attribut **NCE** est une clé primaire c'est à dire un attribut unique qui identifie chaque étudiant.
- Dans la relation *Cours*, c'est **codeCours** est une clé primaire.

Clé candidate :

- Supposons qu'en plus de NCE, on puisse également identifier un étudiant par la combinaison ***(Nom, Prenom)*** — si ces deux attributs ensemble sont uniques dans la relation. Alors, *(Nom, Prenom)* est une **clé candidate** alternative à **NCE**. En pratique, on choisit souvent une clé simple NCE comme clé primaire.

Clé étrangère :

- Dans la relation *Inscription*, les colonnes NCE et codeCours sont des **clés étrangères** dans la relation Inscription :
    - **NCE** est la clé primaire dans la relation *Étudiant*, ce qui garantit que chaque inscription correspond bien à un étudiant existant.
    - **codeCours** est la clé primaire dans la relation *Cours*.

La base impose que les valeurs de ces clés étrangères soient présentes dans leur relation respective, assurant ainsi la cohérence des données (intégrité référentielle).

Cela signifie que chaque NCE de la table Etudiants et codeCours de la table Cours doivent exister dans la table Inscription.

Les clés étrangères sont le mécanisme par lequel les relations entre les tables sont définies et l'intégrité référentielle est maintenue. Elles garantissent que les liens entre les données sont valides et cohérents.

Exercice



Un attribut représente :

- ☐ Un ensemble de relations
- ☐ Une propriété d’une entité
- ☐ Une ligne d’une table
- ☐ Une dépendance fonctionnelle

Exercice



On souhaite modéliser la relation suivante : **Employé(NumEmp, Nom, Poste, Salaire)**

1. Détermine le domaine de chaque attribut.
2. Identifie la clé primaire.
3. Donne le produit cartésien de deux ensembles D1={A,B} et D2={1,2,3}.

Quelles sont les contraintes d’intégrité ?



Les contraintes d'intégrité sont des règles qui garantissent la validité et la cohérence des données dans une base de données relationnelle. Elles sont définies au niveau du schéma de la base de données et sont appliquées automatiquement par le SGBDR lors des **opérations de mise à jour (insertion, modification, suppression)**. Le respect de ces contraintes est fondamental pour maintenir la qualité des données.

Types de contraintes d’intégrité :

1. **Contraintes d’intégrité de domaine :**

   Une contrainte de domaine spécifie que les valeurs d'un attribut doivent appartenir à un ensemble prédéfini de valeurs (son domaine). Cela inclut le type de données (numérique, chaîne de caractères, date, etc.) et peut également inclure des restrictions sur la plage de valeurs ou le format.

   Exemple :

   Un attribut Age doit être un entier positif.

   Un attribut Statut ne peut prendre que les valeurs 'Actif', 'Inactif', 'Suspendu'.

   Un attribut Prix doit être supérieur à zéro.

   Les contraintes de domaine sont essentielles pour garantir que les données saisies sont valides et conformes aux règles métier.

2. **Contraintes d’entité (clé primaire) :**

   La contrainte d'entité stipule que chaque relation doit avoir une clé primaire, et que les valeurs de cette clé primaire doivent être uniques et non NULL pour chaque tuple. Cette contrainte assure l'identification unique de chaque entité dans la base de données.

   Exemple : Dans la table Produits avec ID_Produit comme clé primaire :

ID_Produit	Produit	Prix
P001	Ordinateur	120 000
P002	Souris	5000

- Toute tentative d'insérer un nouveau produit avec ID_Produit = 'P001' échouera car cette valeur existe déjà (violation d'unicité).
- Toute tentative d'insérer un produit avec ID_Produit = NULL échouera (violation de non-nullité).

  La contrainte d'entité est la pierre angulaire de l'intégrité des données, garantissant que chaque enregistrement est identifiable de manière unique et complète.

3. **Contraintes référentielles (clé étrangère) :**

La contrainte référentielle (ou intégrité référentielle) garantit que toute valeur de clé étrangère dans une table (table enfant) doit correspondre à une valeur existante de la clé primaire dans la table référencée (table parent), ou être NULL (si l'attribut de clé étrangère n'est pas défini comme NOT NULL).

Exemple : Tables Departement (table parent) et Employe (table enfant) :

Table Departement :

ID_Dept	Nom_Dept
D10	Ventes
D20	Marketing

Table Employe :

ID_Employe	Nom_Employe	ID_Dept
E001	Amani	D10
E002	Mamadou	D20

- Une tentative d'insérer un employé avec ID_Dept = 'D30' échouera si D30 n'existe pas dans la table Départements .
- Une tentative de supprimer le département D10 échouera si des employés y sont encore affectés (sauf si une règle de suppression en cascade est définie).

4. Contraintes générales :

Une contrainte générale permet de définir des règles plus complexes qui ne sont pas couvertes par les contraintes d'entité, référentielles ou de domaine simples. Elles peuvent impliquer plusieurs attributs d'une même table.

Exemple :

Dans une table Commandes , la Date_Livraison ne peut pas être antérieure à la Date_Commande .

Les contraintes générales offrent une grande flexibilité pour appliquer les règles métier spécifiques et complexes.

Exercice



Une contrainte d’intégrité de domaine concerne :

- ☐ Le type et la validité des valeurs d’un attribut
- ☐ La relation entre deux tables
- ☐ L’unicité des clés primaires
- ☐ La suppression en cascade

Exercice



On considère les relations suivantes :

Département(ID_Dept, Nom_Dept)

Employé(ID_Emp, Nom_Emp, ID_Dept)

1. Quelle contrainte garantit qu'un employé appartient toujours à un département existant ?
2. Que se passe-t-il si on supprime un département encore référencé ?
3. Propose une solution technique pour éviter l'erreur.

Qu'est-ce que la dépendance fonctionnelle ?



La normalisation est un processus essentiel dans la conception des bases de données relationnelles. Son objectif principal est de réduire la redondance des données et d'éliminer les anomalies d'insertion, de suppression et de modification, garantissant ainsi l'intégrité et la cohérence des informations. Ce processus s'appuie sur le concept de dépendances fonctionnelles.

1. Dépendances Fonctionnelles (DF)

Définition:

Une **relation** en bases de données est représentée sous la forme `R(A, B, C)` où **A**, **B** et **C** sont des **attributs** (ou des ensembles d'attributs).

Exemple : Une relation Employe (Numero, Nom, Poste) où :

- A=Numero
- B=Nom
- C=Poste

On dit que **B est fonctionnellement dépendant de A** si :

Pour **tous tuples** $<a_1, b_1, c_1>$ et $<a_2, b_2, c_2>$ de la relation `R`,

si $a_1 = a_2$ alors nécessairement $b_1 = b_2$.

Autrement dit, **la valeur de A détermine de manière unique la valeur de B**.

Notation :

La dépendance fonctionnelle est notée : **A→B**

- Cela se lit :
    - "A détermine B", ou
    - "B est fonctionnellement dépendant de A".

Interprétation simplifiée :

Si on connaît la valeur de A, alors on peut **trouver une seule et unique valeur de B** dans la relation.

Exemple1 : Supposons la relation suivante : `Etudiant(numEt, nom, dateNaiss)`

numEt	nom	dateNaiss
101	Diarra	2001-05-12
102	Koffi	2000-11-03
101	Diarra	2001-05-12

Dans cette relation :

- `numEt → nom` signifie que le **numéro d'étudiant détermine son nom**.
- Autrement dit, deux étudiants ayant le **même numéro** (`numEt`) auront **le même nom** (`nom`).
- `numEt = 101` correspond **toujours à** `nom = Diarra`. Donc, on peut dire : numEt→nom

Exemple2 :

Considérons la relation **Employé(Numero, Nom, Poste)** :

- Numero→Nom : Le numéro d'employé détermine son nom (un employé a un seul nom).
- Numero→Poste : Le numéro détermine aussi le poste (un employé a un seul poste).

Schéma :

Numero (A)	**Nom (B)**	**Poste (C)**
101	Alice	Manager
102	Denis	Dev

si deux tuples ont le même **Numero**, alors leur **Nom** et **Poste** seront identiques.

A→B et A→C sont valides.

Attention

- Une dépendance fonctionnelle **n'est pas une implication logique**, mais une **contrainte sur les données** observées dans la relation.
- Une **même valeur de A** ne doit **jamais être associée à plusieurs valeurs différentes de B**.

Contre-Exemple :

Si on avait une relation **Employe_Departement (Nom, Département)** où un employé peut travailler dans plusieurs départements :

Nom	**Departement**
Rosine	RH
Rosine	Marketing

La dépendance fonctionnelle **Nom → Département** est fausse, car une valeur de Nom (Rosine) peut correspondre à plusieurs valeurs de Département (RH, Marketing). La DF n'est pas valide.

Une **dépendance fonctionnelle** exprime une contrainte d'unicité : *"Une valeur de A détermine une seule valeur de B ."*

C'est un concept fondamental pour la modélisation et l'optimisation des bases de données relationnelles.

Propriétés des dépendances fonctionnelles :

- **Règles d'amstrong :**

Règle	**Formulation**	**Exemple**
Réflexivité	Si Y est un sous-ensemble de X alors X → Y.	(id, nom) → nom.

Augmentation	Si X → Y alors XZ → YZ.	id → nom implique (id, département) → (nom, département).
Transitivité	Si X → Y et Y → Z alors X → Z.	id → email et email → université implique id → université.
Décomposition	Si X → YZ alors X → Y et X → Z.	id → (nom, filière) implique id → nom et id → filière.
Union	Si X → Y et X → Z alors X → YZ.	id → nom et id → filière implique id → (nom, filière).
Pseudo-transitivité	Si X → Y et YW → Z alors XW → Z.	matricule → département et (département, directeur) → budget implique (matricule, directeur) → budget.

Dépendance Fonctionnelle Élémentaire (DFE):

Une dépendance fonctionnelle X→Y est dite **élémentaire** si :

Il **n'existe pas de sous-ensemble strict** $X' \subset X$ tel que $X' \rightarrow Y$.

En d'autres termes :

- La partie gauche X est **minimale**.
- On ne peut pas retirer un attribut de X sans perdre la dépendance.

Exemple:

Considérons une relation : **Etudiant (NumE, Nom, Cours, Note)**

- **(NumE, Cours)→Note**
  - Ici, pour identifier la **Note**, on a besoin du couple `(NumE, Cours)`.
  - Si on enlève `Cours`, alors `NumE→ Note` (faux, car un étudiant peut avoir plusieurs notes selon les cours).
  - Si on enlève `NumE`, alors `Cours → Note` (faux, car plusieurs étudiants dans le même cours n'ont pas la même note).

    Donc, cette DF est **élémentaire**.

Contre-exemple (non élémentaire)

Toujours sur la même relation :

- **(NumE,Nom)→Cours**
  - Ici, `Nom` n'est pas nécessaire car `NumE` suffit à identifier un étudiant et donc ses cours.
  - Donc la dépendance peut être simplifiée en NumE→Cours.

    Ce n'est donc **pas une DFE** (car la partie gauche n'est pas minimale).

Importance des DFE :

- Les DFE sont utilisées pour obtenir une **couverture minimale** d'un ensemble de dépendances fonctionnelles.
- Elles permettent de simplifier les dépendances en supprimant les attributs inutiles.
- Cela facilite la normalisation (jusqu'à la forme normale de Boyce-Codd – BCNF).

Dépendance Fonctionnelle Directe (DFD):

Une dépendance fonctionnelle X→Y, dans une relation R, est dite **directe** si :

Il n'existe pas d'attribut Z de R tel que : X→Z et Z→Y

C'est-à-dire que :

- X→Y est une **dépendance immédiate**.
- Elle **n'est pas obtenue par transitivité**.

Exemple:

Considérons la relation: **R(NumE, Cours, Prof, Note)**

Dépendances :

1. NumE→Note : Pas forcément vrai (un étudiant a plusieurs notes selon les cours).
2. (NumE,Cours)→Note (la note dépend directement de l'étudiant et du cours).
3. Cours→Prof (chaque cours dépend directement d'un seul professeur).
4. NumE→Nom (le matricule identifie directement le nom de l'étudiant).
5. Cours→Prof et Prof→Departement donnent Cours→Departement mais cette dépendance est **indirecte** (car obtenue par transitivité).

Donc :

- Cours→Prof est **directe**.
- Prof→Departement est **directe**.
- Mais Cours→Departement est **indirecte** (car Cours→Prof et Prof→Departement).

3. Contre-exemple (non direct)

Toujours sur la relation **R(NumE, Cours, Prof, Departement, Note)** :

- On a :
    - Cours→Prof
    - Prof→Departement

  Donc par transitivité :
- Cours→Departement

  Ici, Cours→Departement **n'est pas une DFD** car elle est obtenue via un attribut intermédiaire Prof.

Importance des DFD

- Les DFD permettent d'identifier les **vraies dépendances de la base**, celles qui ne sont pas dérivées par transitivité.
- Elles sont cruciales pour la **normalisation** : en particulier lorsqu'on veut passer en **3NF (3ème Forme Normale)** ou en **BCNF**.
- Elles aident à mieux comprendre la structure logique et à éviter les redondances.

Exercice



Une dépendance fonctionnelle A → B signifie :

- ☐ A dépend de B
- ☐ A et B sont indépendants
- ☐ B dépend de A
- ☐ A et B ont le même domaine

Exercice



Soit la relation **Etudiant(numEt, nom, filière, moyenne)**.

1. Indique si la dépendance suivante est valide : numEt → nom.
2. La dépendance nom → filière est-elle vraie ? Justifie.
3. Donne un exemple de dépendance fonctionnelle élémentaire.

Qu'est-ce que la normalisation?

Définition

La normalisation est un **processus de décomposition** des schémas de relations (tables) pour éliminer les redondances (la duplication inutile de données) et les anomalies :

- anomalie d'insertion : impossibilité d'insérer certaines données sans en ajouter d'autres ;
- anomalie de suppression : perte d'informations en supprimant un tuple ;
- anomalie de modification : incohérences lors de la mise à jour d'une donnée présente en plusieurs exemplaires.

But : réduire les incohérences et améliorer l'intégrité des données.

Classification des relations selon les formes normales

La classification se fait en fonction des dépendances fonctionnelles (DF) existant entre les attributs d'une relation.

Il existe plusieurs **formes normales (FN)**, chacune correspondant à un niveau de qualité de conception :

1. **1FN** – Première forme normale
2. **2FN** – Deuxième forme normale
3. **3FN** – Troisième forme normale
4. **BCNF** – Forme normale de Boyce-Codd
5. **4FN** – Quatrième forme normale
6. **5FN** – Cinquième forme normale

On dit qu'un schéma a un **degré de normalité** : plus il est élevé, plus la redondance et les anomalies de MAJ sont réduites.

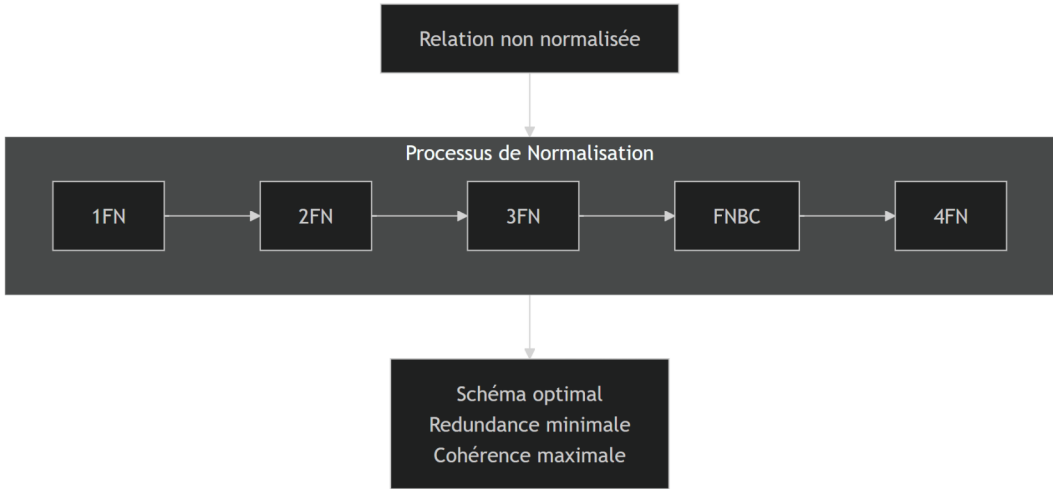


Image 2 les étapes de la normalisation

1. Première Forme Normale (1FN)

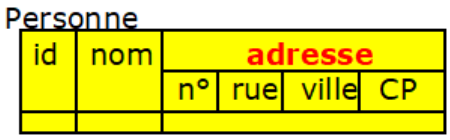
Définition :

Une relation est en **1FN** si tous ses attributs sont **atomiques** (indivisibles). La notion d'agrégat ou de groupe répétitif (ensemble, liste, tableau) est interdite.

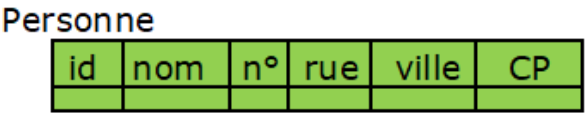
Technique de normalisation :

- **Éclatement d'un attribut composé** : remplacer l'attribut par les attributs simples correspondants.

  Exemple : une relation **Personne** contenant l'attribut adresse



  Il faut décomposer cet attribut



2. Deuxième Forme Normale (2FN)

Définition :

Une relation est en **2FN** si :

- Elle est en **1FN**.
- Tout attribut n'appartenant à **aucune clé** dépend de **la totalité de la clé primaire** (et non d'une partie de celle-ci). Cela élimine les *dépendances partielles*.
- **Remarque importante :** Si la clé primaire est composée d'un seul attribut (clé simple), la relation est automatiquement en 2FN dès qu'elle est en 1FN.
- **Exemple :** Soit la relation `Ventes(`**`Num_Produit`**`,` **`Num_Magasin`**`, Qté_Vendue, Adresse_Magasin)`.
  - La clé est (`Num_Produit`, `Num_Magasin`).
  - `Qté_Vendue` dépend de la clé entière (combien de *ce produit* a été vendu dans *ce magasin*).
  - `Adresse_Magasin` ne dépend que de `Num_Magasin` (une partie de la clé). C'est une **dépendance partielle**. La relation n'est pas en 2FN.
- **Technique de normalisation :** Il faut décomposer la relation.
  - `Ventes(`**`Num_Produit`**`,` **`Num_Magasin`**`, Qté_Vendue)`
  - `Magasins(`**`Num_Magasin`**`, Adresse_Magasin)`

3. Troisième Forme Normale (3FN)

Définition :

Une relation est en **3FN** si :

- Elle est en **2FN**.
- Tout attribut n'appartenant à **aucune clé** ne dépend **pas** d'un attribut non clé. Cela élimine les *dépendances transitives*.
- **Méthode pour vérifier la 3FN :**
    1. Identifier toutes les clés candidates (clés minimales) de la relation.
    2. Lister tous les attributs qui n'appartiennent à **aucune** de ces clés (les attributs non clés).
    3. Pour chaque DF `X → A` où `A` est un attribut non clé, vérifier que `X` est **une superclé** (contient une clé candidate). Si ce n'est pas le cas pour une DF, la relation n'est pas en 3FN.
- **Exemple :** Soit la relation `Commandes(`**`Num_Commande`**`, Date,`**`Client_ID`**`, Nom_Client, Ville_Client)`.
    - Clé : `Num_Commande`.
    - `Client_ID` et `Nom_Client` sont des attributs non clés.
    - On a les DF : `Num_Commande → Client_ID` et `Client_ID → Nom_Client, Ville_Client`.
    - `Nom_Client` et `Ville_Client` dépendent *transitivement* de la clé `Num_Commande` via l'attribut non clé `Client_ID`. La relation n'est pas en 3FN.
- **Technique de normalisation :** Il faut décomposer pour supprimer la dépendance transitive.
    - `Commandes(`**`Num_Commande`**`, Date,`**`Client_ID`**`)`
    - `Clients(`**`Client_ID`**`, Nom_Client, Ville_Client)`

Remarque :

La maîtrise des trois premières formes normales est essentielle pour tout concepteur de base de données. Elles permettent de structurer les données de manière logique, efficace et fiable, formant la base d'un système d'information cohérent.

Exercice



Une relation est en 1FN si :

- ☐ Il n’y a pas de clé primaire
- ☐ Les attributs sont atomiques
- ☐ Les valeurs sont nulles
- ☐ Les tuples sont triés

Exercice



On dispose de la table suivante :

Commandes(Num_Commande, Date, Client_ID, Nom_Client, Ville_Client)

1. Identifie les anomalies présentes.
2. Propose la décomposition pour obtenir la 3FN.

Comment transformer le MCD en Modèle Logique de Données Relationnel (MLDR) ?



Le **Modèle Logique de Données Relationnel (MLDR)** est une représentation abstraite de la structure des données d'une base de données, basée sur les concepts du **modèle relationnel**.

Il intervient après la phase de **conception conceptuelle** (généralement matérialisée par un **Modèle Conceptuel de Données (MCD)** ou un **diagramme de classes UML**).

Son objectif principal est de spécifier comment les données seront stockées dans la future base de données relationnelle, en définissant précisément les tables, leurs colonnes, les clés et les contraintes.

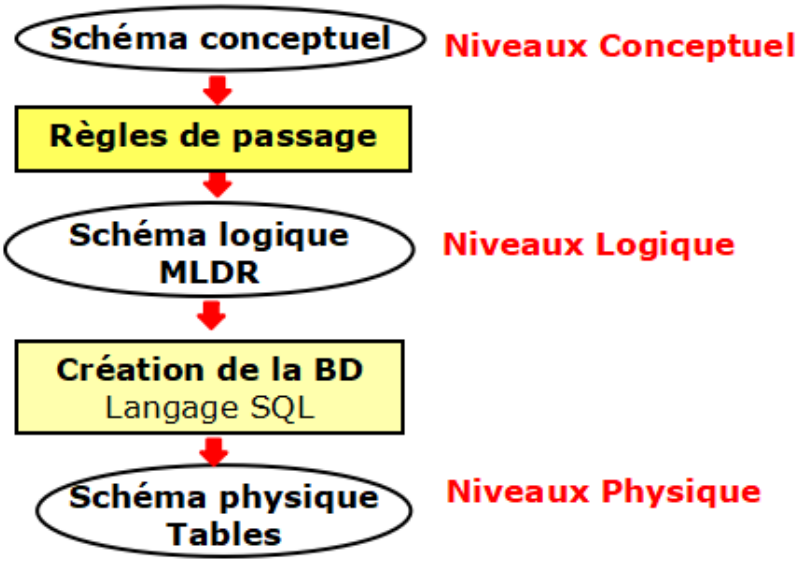


Image 3 Démarche de création du MLDR

Règles de passage du MCD au MLDR :

Le passage du **Modèle Conceptuel de Données (MCD)** au **Modèle Logique de Données Relationnel (MLDR)** repose sur un ensemble de règles systématiques. Ces règles permettent de traduire les entités, associations et héritages en relations adaptées au paradigme relationnel.

Règle 1 : Chaque entité devient une relation

- Chaque **entité** du MCD est transformée en **table** dans le MLDR.
- L'**identifiant de l'entité** devient la **clé primaire** de la table correspondante.
- Les **attributs de l'entité** deviennent les **colonnes** de la table.

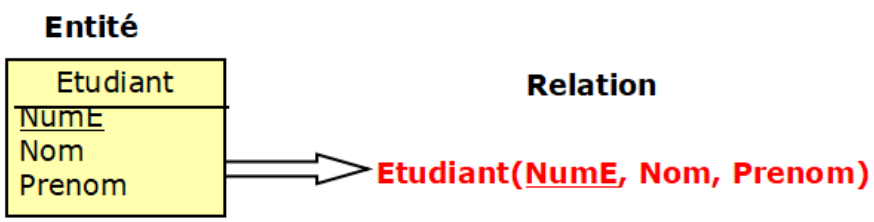


Image 4 Transformation d'une entité en une relation

Règle 2 : Association de type **1:n** (relation père-fils)

- Le **père** est l'entité du côté de la cardinalité maximale **n** ;
- Le **fils** est l'entité du côté de la cardinalité maximale **1** ;
- L'**identifiant du père** (clé primaire) **migre dans le fils** en tant que **clé étrangère**.
- Les **attributs de l'association** sont placés dans la table du fils.

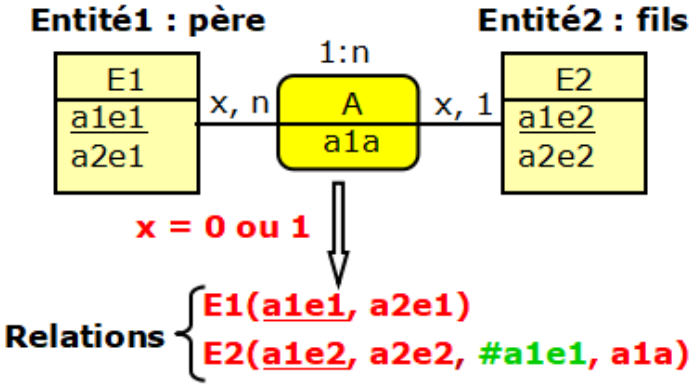


Image 5 Transformation d'une association de type père-fils

Règle 3 : Association de type n:n

- Une association de type **n:n** devient une **nouvelle relation** ;
- les propriétés de la nouvelle relation sont les propriétés de l'association et les identifiants des entités qui participent à l'association ;
- La **clé primaire** de cette relation est la **concaténation des identifiants des entités participantes**.

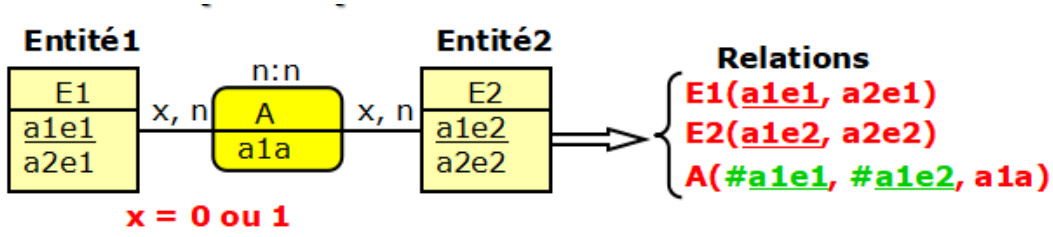


Image 6 Transformation d'une association de type n :n

Règle 4 : Association réflexive

- Une **association réflexive** (une entité liée à elle-même) se traduit selon les cardinalités :
  - Si c'est une relation **1:n**, on applique la règle 2 (clé étrangère qui migre).
  - Si c'est une relation **n:n**, on applique la règle 3 (création d'une table supplémentaire).

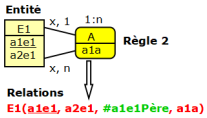


Image 7 Transformation d'une association réflexive de type 1 :n

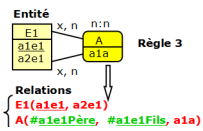


Image 8 Transformation d'une association réflexive de type n :n

Règle 5 : Association de type 1:1

- Dans le cas d'une relation **1:1**, la clé primaire d'une entité **migre dans l'autre entité** comme clé étrangère.
- Le choix de la migration se fait en fonction :
  - de la **cardinalité obligatoire** (migre du côté optionnel vers le côté obligatoire),
  - ou des **critères de conception pratique** (minimiser les valeurs nulles).

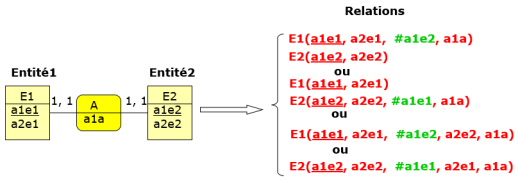


Image 9 Exemple 1 de la transformation d'une association de type 1 :1

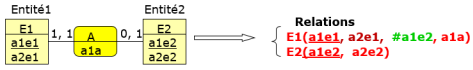


Image 10 Exemple 2 de la transformation d'une association de type 1 :1

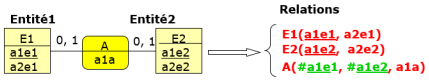


Image 11 Exemple 3 de la transformation d'une association de type 1 :1

Règle 6 : héritage d'entité

- l'identifiant de l'entité générique devient la clé des entités spécialisées ;
- les entités spécialisées n'ont pas d'identifiants propres.

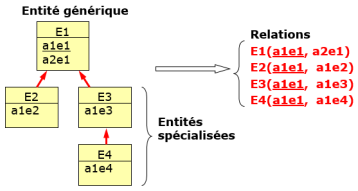


Image 12 Transformation d'un héritage d'entité

Exercice



Chaque entité du MCD devient :

- ☐ Un domaine
- ☐ Un attribut
- ☐ Une table relationnelle
- ☐ Une clé étrangère

Exercice



À partir du MCD suivant :

Etudiant (matricule, nom)

Cours (codeCours, libellé)

Inscrire (date_inscription) : relie Étudiant et Cours (N:N)

Transforme ce MCD en MLDR.

Conclusion

Le modèle logique de données relationnel traduit la structure conceptuelle (MCD) dans un cadre plus concret, directement exploitable par les SGBDR.

La compréhension des relations, clés, contraintes, dépendances fonctionnelles et formes normales garantit la cohérence et la robustesse des bases de données.

Enfin, la transformation du MCD en MLDR constitue la passerelle essentielle entre la modélisation conceptuelle et l'implémentation physique, assurant une base solide pour tout système d'information moderne.